МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В. И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №4

по дисциплине «Вычислительная математика»

Тема: Метод хорд

Студент гр. 4342 _____ Тиссен М.Ю.

Преподаватель _____ Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Цель лабораторной работы - изучить численное решение нелинейного уравнения f(x) = 0 методом хорд, реализовать подпрограммы вычисления функции и округления Round, найти корень уравнения для варианта 18 с заданной точностью, а также исследовать скорость сходимости и чувствительность метода к округлению значений функции.

Задание.

Пусть найден отрезок [a, b], на котором функция f (x) меняет знак. Для определенности положим f (a)>0, f (b)<0. В методе хорд процесс итераций состоит в том, что в качестве приближений к корню уравнения f (x) = 0 принимаются значения c0, c1, . . . точек пересечения хорды с осью абсцисс, как это показано на рис.1.

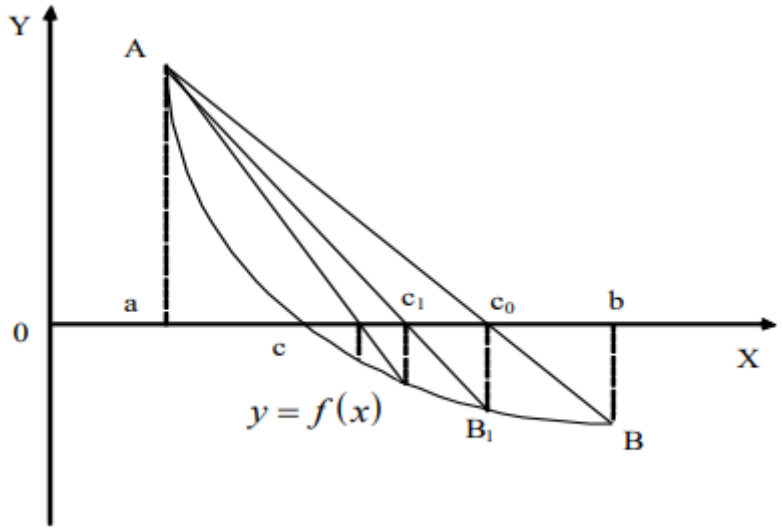

Рис. 1

Сначала находится уравнение хорды AB:

$$\frac{y-f(a)}{f(b)-f(a)} = \frac{x-a}{b-a}$$

Для точки пересечения ее с осью абсцисс (x=c0, y=0) получается уравнение

$$c_0 = a - \frac{b-a}{f(b)-f(a)} f(a)$$

Далее сравниваются знаки величин f (a) и f (c0) и для рассматриваемого случая оказывается, что корень находится в интервале (a, c0), так как f (a) f (c0)<0. Отрезок [c0,b] отбрасывается. Следующая итерации состоит в определении нового приближения c1 как точки пересечения хорды AB1 с осью абсцисс и т.д. Итерационный процесс продолжается до тех пор, пока значение f (cn) не станет по модулю меньше заданного числа ε (см. подраздел 3.1).

Алгоритмы методов бисекции и хорд похожи, однако метод хорд в ряде случаев дает более быструю сходимость итерационного процесса, причем успех его применения, как и метода бисекции, гарантирован.

В лабораторной работе №4 предлагается, используя программы - функции HORDA и Round из файла methods.cpp (файл заголовков metods.h, директория LIBR1), найти корень уравнения f (x) = 0 с заданной точностью Eps методом хорд, исследовать скорость сходимости и обусловленности метода.

Для данной работы, как и для лабораторной работы №3 задаются индивидуальные варианты нелинейных уравнений (см. подраздел 3.6).

Порядок выполнения лабораторной работы №4:

1) Графически или аналитически отделить корень уравнения f (x) = 0 (т.е. найти отрезки [Left, Right], на которых функция f (x) удовлетворяет условиям применимости метода).

2) Составить подпрограмму - функцию вычисления функции f (x) , предусмотрев округление значений функции с заданной точностью Delta с использованием программы Round.

3) Составить головную программу, вычисляющую корень уравнения f (x) = 0 и содержащую обращение к подпрограмме f(x), HORDA, Round и индикацию результатов.

4) Провести вычисления по программе. Теоретически и экспериментально исследовать скорость сходимости и обусловленность метода.

Вариант 18: $f(x) = e^{(1/x^2)} - \ln(x)$.

Выполнение работы.

1. Отделение (изоляция) корня уравнения.

Для варианта 18 рассматривается функция

$$f(x) = e^{(1/x^2)} - \ln(x), \quad x > 0.$$

Функция определена и непрерывна при $x > 0$. Для дополнительного обоснования единственности корня рассмотрим производную:

$$f'(x) = -2e^{(1/x^2)}/x^3 - 1/x.$$

При $x > 0$ обе слагаемые отрицательны, следовательно, $f'(x) < 0$ и функция строго убывает на всей области определения. Поэтому на любом интервале, где наблюдается смена знака, корень будет единственным.

Проверим значения функции на концах отрезка [3; 4]:

$$f(3) = e^{(1/9)} - \ln(3) \approx 0{,}018906780074 > 0,$$

$$f(4) = e^{(1/16)} - \ln(4) \approx -0{,}321799902202 < 0.$$

Так как $f(3)*f(4) < 0$, то по теореме Больцано на интервале (3; 4) существует корень уравнения $f(x) = 0$. С учётом монотонного убывания функции этот корень единственный, поэтому в качестве начального интервала для метода хорд принимаем [Left; Right] = [3; 4].

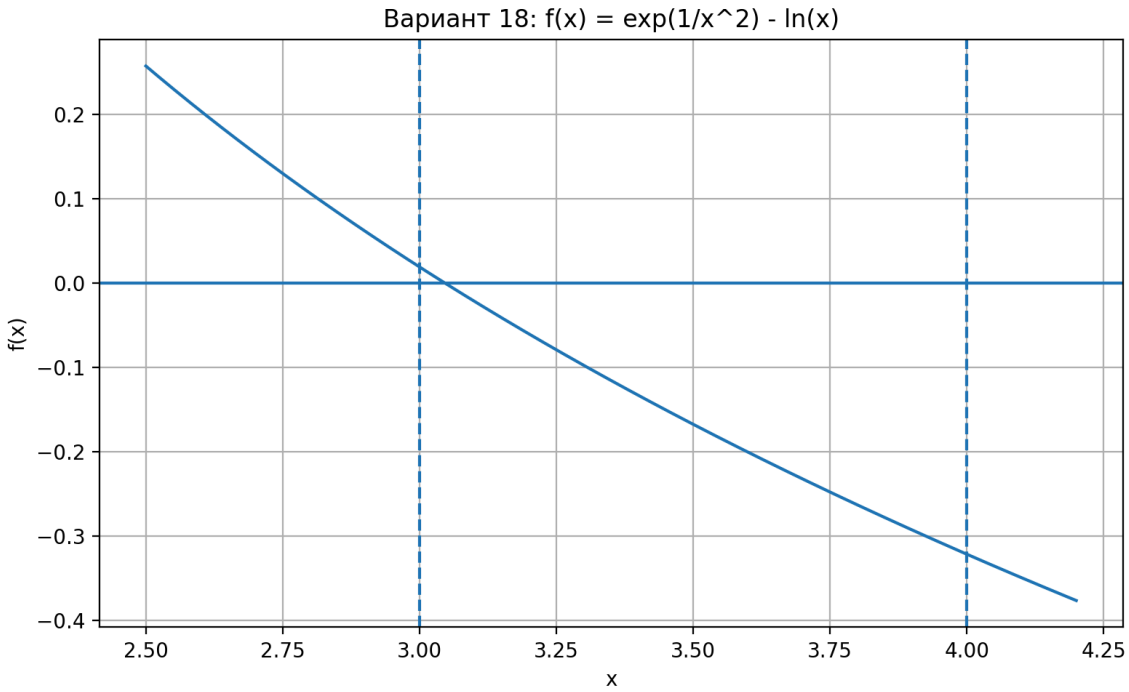

Рисунок 2 — График функции f(x) = e^(1/x²) − ln(x) и отрезок изоляции корня [3; 4].

2. Подпрограмма вычисления функции f(x).

В файле main.cpp функция варианта реализована отдельной подпрограммой f(x). В ней учитывается область определения задачи: при $x \le 0$ вычисление логарифма невозможно, поэтому в программе генерируется исключение. На рабочем интервале [3; 4] этот контроль нужен скорее как защита от неверных входных данных.

Для исследования обусловленности дополнительно используется подпрограмма Round(value, Δ), которая округляет значение функции до ближайшего числа, кратного Δ. При Δ = 0 округление не выполняется. Таким образом, в вычислениях применяется не только точная функция f(x), но и её модель с ограниченной точностью:

$$f_\Delta(x) = \mathrm{Round}(f(x), \Delta).$$

Такой подход позволяет проверить, насколько устойчив метод хорд к ошибкам в вычислении значений функции. Особенно важно это в формуле метода, где используются сразу f(a) и f(b): если округление слишком грубое, то интервал может перестать удовлетворять условию смены знака.

3. Подпрограммы Round и HORDA.

В файле methods.cpp реализована функция Round и основная процедура HORDA. Метод хорд строит последовательность приближений как точки пересечения хорды, проведённой через точки (a, f(a)) и (b, f(b)), с осью Ox.

$$c = a - f(a)\cdot(b - a) / (f(b) - f(a)).$$

После вычисления очередного приближения c определяется знак f(c). Затем сохраняется тот подотрезок, на котором по-прежнему есть смена знака. В программе используются два критерия остановки: $|f(c)| < Eps$ или $|b - a| < 2\cdot Eps$. Результат работы метода возвращается через структуру HordaResult, которая содержит найденное приближение корня, текущий интервал, число итераций и признак успешной сходимости.

Для варианта 18 метод работает устойчиво, потому что функция на [3; 4] монотонна и достаточно гладкая. На практике это приводит к очень быстрому уменьшению невязки и малому числу итераций.

4. Головная программа и организация экспериментов.

Главная программа задаёт интервал [3; 4], вычисляет опорный корень методом хорд с высокой точностью ($Eps = 10^{-14}$, $\Delta = 0$), а затем проводит два набора экспериментов.

Первый набор используется для исследования скорости сходимости. Значение Eps меняется в диапазоне от 10^{-1} до 10^{-6}, после чего для каждого запуска сохраняются число итераций, найденный корень и длина текущего интервала. Данные записываются в файл eps_iter_profile.csv.

Второй набор посвящён обусловленности метода. Для фиксированной точности $Eps = 10^{-8}$ берутся разные значения Δ, после чего сравнивается найденный корень $x\Delta$ с эталонным значением xref. Результаты записываются в файл delta_impact_profile.csv. Построение графиков выполняется отдельным скриптом plot.py.

5. Результаты вычислительных экспериментов.

Опорный расчёт без округления дал значение

$$xref \approx 3{,}045936104786.$$

Для получения этого значения методу хорд потребовалось 7 итераций при Eps = 10^{-14}. Ниже приведены результаты основных экспериментов.

5.1. Исследование скорости сходимости.

В таблице 1 показано, как меняется число итераций при уменьшении требуемой точности Eps. Видно, что для данного уравнения метод хорд сходится очень быстро: для Eps = 10^{-6} оказалось достаточно трёх итераций.

Eps	N	Корень x*	|f(x*)|
$1\cdot10^{-1}$	1	3,05549284	$3{,}88\cdot10^{-3}$
$1\cdot10^{-2}$	1	3,05549284	$3{,}88\cdot10^{-3}$
$1\cdot10^{-3}$	2	3,04603945	$4{,}21\cdot10^{-5}$
$1\cdot10^{-4}$	2	3,04603945	$4{,}21\cdot10^{-5}$
$1\cdot10^{-5}$	3	3,04593722	$4{,}55\cdot10^{-7}$
$1\cdot10^{-6}$	3	3,04593722	$4{,}55\cdot10^{-7}$

Таблица 1 — Результаты исследования скорости сходимости метода хорд.

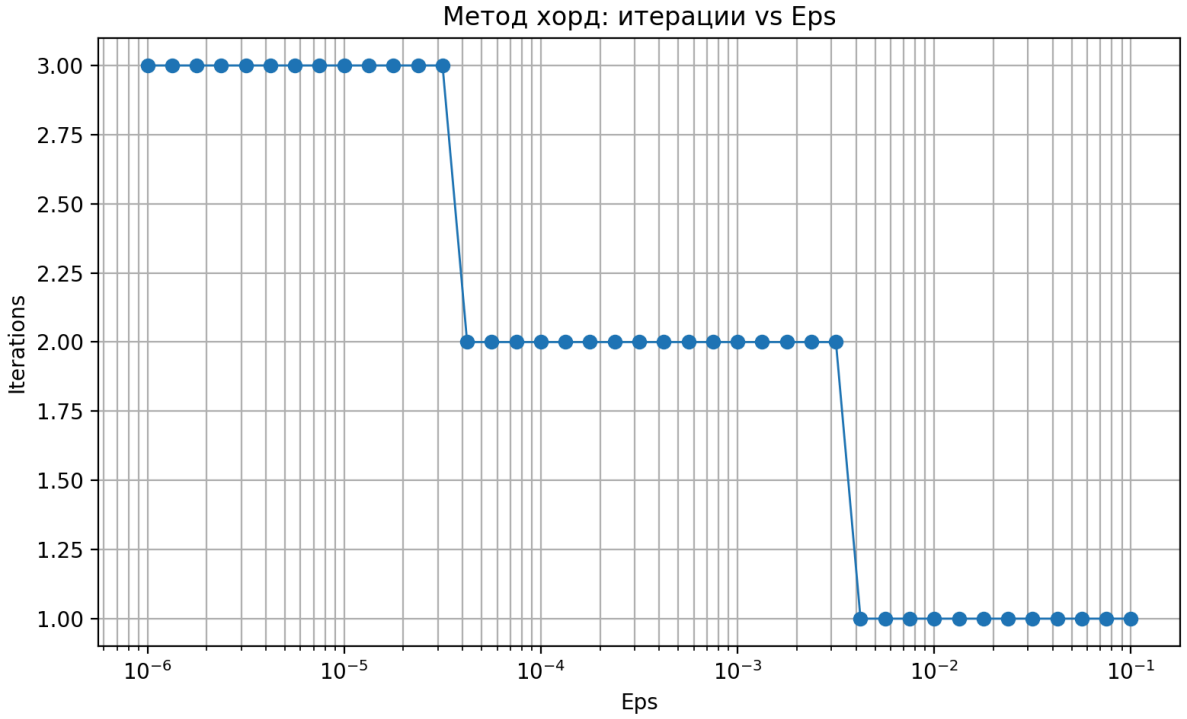

Рисунок 3 — Зависимость числа итераций метода хорд от требуемой точности Eps.

По графику видно ступенчатое изменение числа итераций. Это связано с тем, что на рассматриваемом интервале функция близка к линейной, поэтому один и тот же номер итерации обеспечивает сразу целый диапазон значений Eps. По сравнению с методом бисекции из предыдущей лабораторной работы метод хорд для данного уравнения оказался заметно быстрее.

5.2. Исследование чувствительности к округлению.

В таблице 2 приведены результаты при фиксированной точности Eps = 10^{-8} и разных значениях Δ. Абсолютная ошибка вычислялась по формуле |xΔ − xref|. При Δ = 0,1 метод не запускается, потому что после округления получаем Round(f(3), 0,1) = 0, а условие f(a)·f(b) < 0 перестаёт выполняться.

Δ	**N**	**Корень Δx**	**Абс. ошибка**	**Комментарий**
0	4	3,04593612	$1{,}21\cdot10^{-8}$	без округления
$1\cdot10^{-1}$	0	—	—	метод не стартует
$1\cdot10^{-2}$	2	3,03921569	$6{,}72\cdot10^{-3}$	ошибка заметна
$1\cdot10^{-3}$	2	3,04602831	$9{,}22\cdot10^{-5}$	ещё грубо
$1\cdot10^{-4}$	2	3,04598505	$4{,}89\cdot10^{-5}$	ошибка меньше
$1\cdot10^{-5}$	3	3,04593520	$9{,}01\cdot10^{-7}$	близко к эталону
$1\cdot10^{-6}$	4	3,04593591	$1{,}99\cdot10^{-7}$	ошибка мала
$1\cdot10^{-7}$	4	3,04593619	$8{,}36\cdot10^{-8}$	очень мала
$1\cdot10^{-8}$	5	3,04593610	$7{,}51\cdot10^{-9}$	почти эталон

Таблица 2 — Влияние округления значений функции на результат метода хорд.

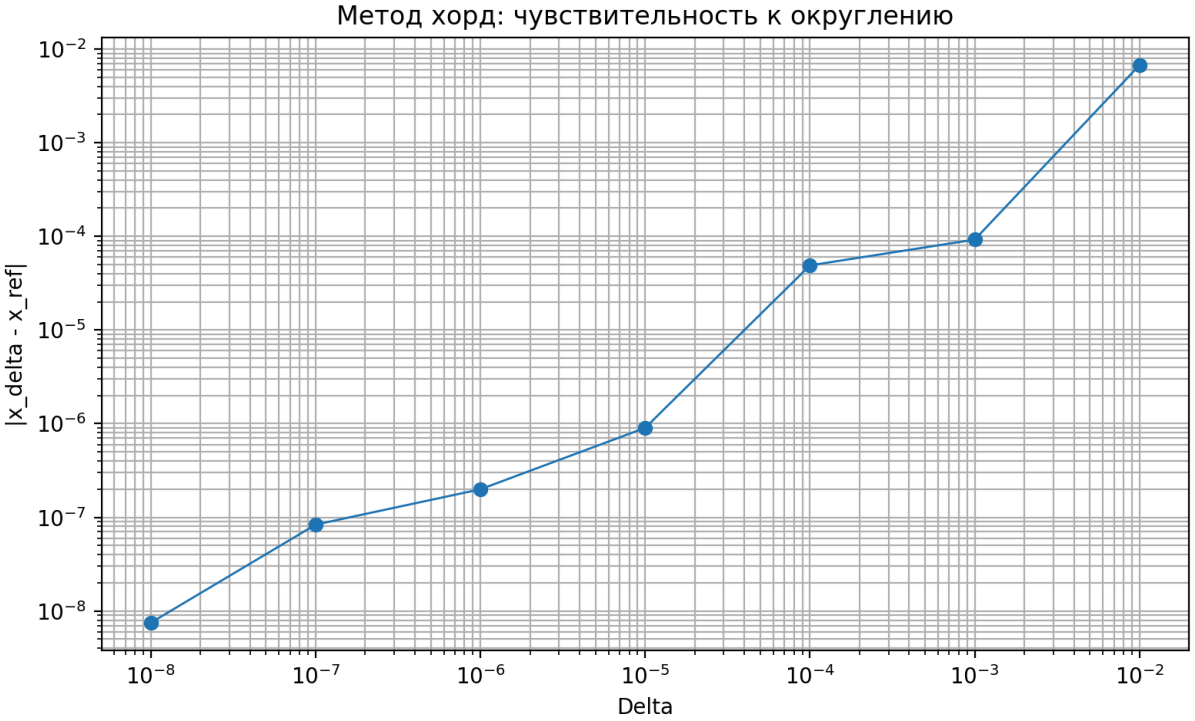

Рисунок 4 — Зависимость абсолютной ошибки |xΔ − xref| от параметра Δ.

С уменьшением Δ абсолютная ошибка в целом снижается. Уже при $\Delta \leq 10^{-6}$ найденный корень практически совпадает с эталонным значением, поэтому метод хорд для данного варианта можно считать устойчивым к малым погрешностям.

Вывод.

В ходе лабораторной работы был реализован метод хорд для решения нелинейного уравнения варианта 18 $f(x) = e^{(1/x^2)} - \ln(x)$. Корень уравнения был изолирован на интервале [3; 4], причём благодаря монотонному убыванию функции на $x > 0$ удалось обосновать его единственность. Программно реализованы подпрограммы f(x), Round и HORDA, а также автоматическое формирование таблиц и графиков эксперимента.

Численные расчёты показали, что при $Eps = 10^{-6}$ методу достаточно трёх итераций, а при малых Δ результат остаётся близким к эталонному корню $xref \approx 3{,}045936104786$. Следовательно, метод хорд для данной задачи эффективен и устойчив.

ПРИЛОЖЕНИЕ А

Название файла: methods.h

```
#pragma once

#include <functional>

struct HordaResult {
    double x;       // найденное приближение корня
    double a, b;    // текущий отрезок, сохраняющий смену знака
    int iters;      // число выполненных итераций
    bool ok;        // достигнут ли критерий остановки
};

// Функция Round - округление числа до кратного delta.
// Используется для моделирования ошибок вычисления значений функции.
double Round(double value, double delta);

// Метод хорд для поиска корня уравнения f(x) = 0.
// f             - функция одной переменной
// left, right   - границы начального отрезка изоляции
// eps           - требуемая точность корня
// nmax          - предельное число итераций
// delta         - шаг округления значений f(x) (0 = без округления)
HordaResult HORDA(const std::function<double(double)>& f,
                  double left,
                  double right,
                  double eps,
                  int nmax = 1000000,
                  double delta = 0.0);
```

Название файла: methods.cpp

```
#include "methods.h"
#include <cmath>
#include <limits>

// ==============================================================
// Функция Round - округление числа до заданной точности delta
// ==============================================================
double Round(double value, double delta) {
    if (delta <= 0.0) {
        return value;
    }
    return std::round(value / delta) * delta;
}

// ==============================================================
// Метод хорд
// Находит корень уравнения f(x) = 0 на отрезке [left, right]
// ==============================================================
HordaResult HORDA(const std::function<double(double)>& f,
                  double left,
                  double right,
```

```
                double eps,
                int nmax,
                double delta) {
    // Инициализация результата по умолчанию.
    HordaResult result{};
    result.x = std::numeric_limits<double>::quiet_NaN();
    result.a = left;
    result.b = right;
    result.iters = 0;
    result.ok = false;

    double a = left;
    double b = right;

    // Значения функции на концах отрезка с учетом возможного
округления.
    double fa = Round(f(a), delta);
    double fb = Round(f(b), delta);

    // Проверка условия изоляции корня.
    if (!(fa * fb < 0.0)) {
        return result;
    }

    // Основной цикл метода хорд.
    for (int iter = 1; iter <= nmax; ++iter) {
        const double denominator = fb - fa;
        // Если знаменатель обнулился, следующую точку построить
нельзя.
        if (denominator == 0.0) {
            result.x = 0.5 * (a + b);
            result.a = a;
            result.b = b;
            result.iters = iter;
            result.ok = false;
            return result;
        }

        // Точка пересечения хорды с осью Ox.
        const double c = a - fa * (b - a) / denominator;
        const double fc = Round(f(c), delta);

        // Остановка по малому значению функции или по длине отрезка.
        if (std::abs(fc) < eps || std::abs(b - a) < 2.0 * eps) {
            result.x = c;
            result.a = a;
            result.b = b;
            result.iters = iter;
            result.ok = true;
            return result;
        }

        // Оставляем ту часть отрезка, где сохраняется смена знака.
        if (fa * fc < 0.0) {
            b = c;
            fb = fc;
        } else {
            a = c;
```

```
            fa = fc;
        }
    }

    // Если достигнут предел итераций, возвращаем последнее
приближение.
    result.x = 0.5 * (a + b);
    result.a = a;
    result.b = b;
    result.iters = nmax;
    result.ok = false;
    return result;
}
```

Название файла: main.cpp

```
#include <cmath>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <vector>

#include "methods.h"

// ==============================================================
// ВАРИАНТ 18: f(x) = e^(1/x^2) - ln(x)
// Область определения: x > 0
// Интервал изоляции: [3; 4]
// ==============================================================
double f(double x) {
    if (x <= 0.0) {
        throw std::runtime_error("Function is undefined for x <= 0");
    }
    return std::exp(1.0 / (x * x)) - std::log(x);
}

// ==============================================================
// Головная программа
// ==============================================================
int main() {
    const double Left = 3.0;
    const double Right = 4.0;
    const int Nmax = 2000000;

    std::cout << std::fixed << std::setprecision(12);
    std::cout << "Laboratornaya rabota #4. Metod hord.\n";
    std::cout << "Variant 18: f(x) = exp(1/x^2) - ln(x)\n";
    std::cout << "Interval [" << Left << "; " << Right << "]\n";
    std::cout << "f(Left)  = " << f(Left) << "\n";
    std::cout << "f(Right) = " << f(Right) << "\n\n";

    const auto reference = HORDA(f, Left, Right, 1e-14, Nmax, 0.0);
    if (!reference.ok) {
        std::cerr << "Reference root was not obtained.\n";
        return 1;
    }
```

```
    std::cout << "Reference root (eps = 1e-14, delta = 0):\n";
    std::cout << "x* = " << reference.x
              << ", iterations = " << reference.iters << "\n\n";

    std::vector<double> eps_values;
    for (int i = 0; i <= 40; ++i) {
        const double alpha = static_cast<double>(i) / 40.0;
        // Изменяем eps от 1e-1 до 1e-6 по логарифмической шкале.
        const double power = -1.0 - 5.0 * alpha;
        eps_values.push_back(std::pow(10.0, power));
    }

    // ==============================================================
    // Исследование зависимости числа итераций от точности eps
    // ==============================================================
    std::ofstream eps_file("eps_iter_profile.csv");
    eps_file << "Eps;Iterations;Root;IntervalLen;Ok\n";

    std::cout << "Speed of convergence study:\n";
    for (double eps : eps_values) {
        const auto result = HORDA(f, Left, Right, eps, Nmax, 0.0);
        const double interval_len = std::abs(result.b - result.a);

        eps_file << std::setprecision(16)
                 << eps << ';'
                 << result.iters << ';'
                 << result.x << ';'
                 << interval_len << ';'
                 << (result.ok ? 1 : 0) << '\n';

        std::cout << "eps = " << std::setw(12) << eps
                  << "  iterations = " << std::setw(3) << result.iters
                  << "  x = " << result.x << '\n';
    }
    eps_file.close();

    // ==============================================================
    // Исследование чувствительности к ошибкам округления delta
    // ==============================================================
    const double fixed_eps = 1e-8;
    const std::vector<double> delta_values = {
        0.0, 1e-1, 1e-2, 1e-3, 1e-4, 1e-5, 1e-6, 1e-7, 1e-8
    };

    std::ofstream delta_file("delta_impact_profile.csv");
    delta_file << "Delta;Eps;Iterations;Root;AbsErrorToRef;Ok\n";

    std::cout << "\nSensitivity study (fixed eps = 1e-8):\n";
    for (double delta : delta_values) {
        const auto result = HORDA(f, Left, Right, fixed_eps, Nmax,
delta);
        const double error = result.ok
            ? std::abs(result.x - reference.x)
            : std::numeric_limits<double>::quiet_NaN();

        delta_file << std::setprecision(16)
                   << delta << ';'
                   << fixed_eps << ';'
```

```
                    << result.iters << ';'
                    << result.x << ';'
                    << error << ';'
                    << (result.ok ? 1 : 0) << '\n';

        std::cout << "delta = " << std::setw(10) << delta
                  << "  ok = " << result.ok
                  << "  iterations = " << std::setw(3) << result.iters
                  << "  x = " << result.x << '\n';
    }
    delta_file.close();

    return 0;
}
```

Название файла: plot.py

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# ==============================================================
# Функция варианта 18
# ==============================================================
def f(x: np.ndarray) -> np.ndarray:
    return np.exp(1.0 / (x * x)) - np.log(x)

# ==============================================================
# График 0: функция и интервал изоляции корня
# ==============================================================
x = np.linspace(2.5, 4.2, 600)
y = f(x)

plt.figure(figsize=(8, 5))
plt.plot(x, y)
plt.axhline(0.0)
plt.axvline(3.0, linestyle="--")
plt.axvline(4.0, linestyle="--")
plt.xlabel("x")
plt.ylabel("f(x)")
plt.title("Вариант 18: f(x) = exp(1/x^2) - ln(x)")
plt.grid(True)
plt.tight_layout()
plt.savefig("chords_isolation_plot.png", dpi=200)
plt.close()

# ==============================================================
# График 1: число итераций от точности Eps
# ==============================================================
eps_df = pd.read_csv("eps_iter_profile.csv",
sep=";").sort_values("Eps")
plt.figure(figsize=(8, 5))
plt.plot(eps_df["Eps"], eps_df["Iterations"], marker="o", linewidth=1)
plt.xscale("log")
plt.xlabel("Eps")
plt.ylabel("Iterations")
```

```
plt.title("Метод хорд: итерации vs Eps")
plt.grid(True, which="both")
plt.tight_layout()
plt.savefig("chords_eps_curve.png", dpi=200)
plt.close()

# ==============================================================
# График 2: погрешность корня от точности округления Delta
# ==============================================================
delta_df = pd.read_csv("delta_impact_profile.csv", sep=";")
delta_ok = delta_df[(delta_df["Ok"] == 1) & (delta_df["Delta"] >
0)].copy()
delta_ok = delta_ok.sort_values("Delta")

plt.figure(figsize=(8, 5))
plt.plot(delta_ok["Delta"], delta_ok["AbsErrorToRef"], marker="o",
linewidth=1)
plt.xscale("log")
plt.yscale("log")
plt.xlabel("Delta")
plt.ylabel("|x_delta - x_ref|")
plt.title("Метод хорд: чувствительность к округлению")
plt.grid(True, which="both")
plt.tight_layout()
plt.savefig("chords_delta_curve.png", dpi=200)
plt.close()
```